GAUTHER Paul
LE MAIRE Teo

19/01/25

RAPPORT DE PROJET - WEBMAPPING

Création d'une application webmapping en architecture trois-tiers pour l'aide à la prospection d'un commercial indépendant

---

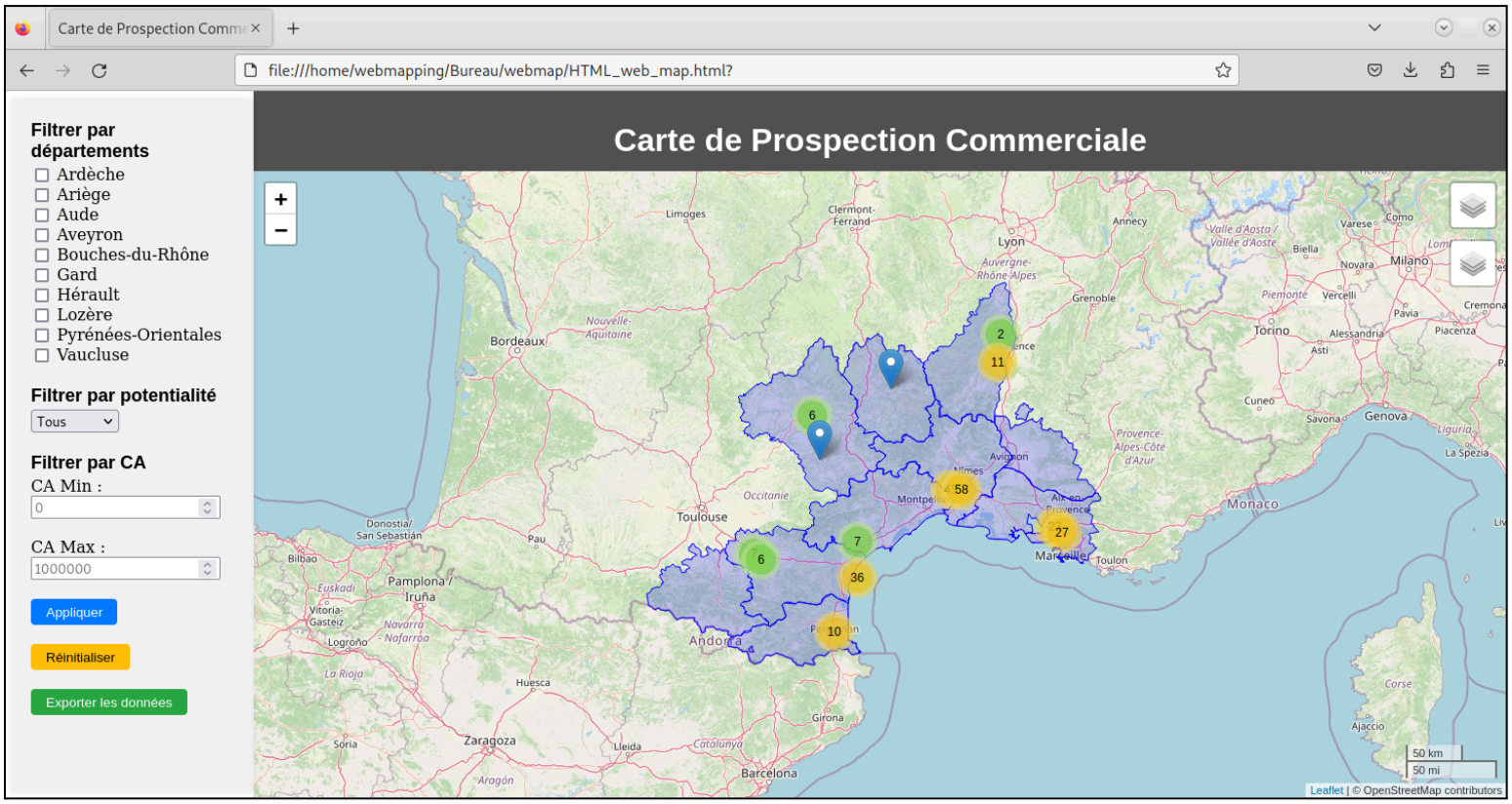

Figure 1 - Capture d'écran de l'interface web cartographique de notre application

1. Introduction

1.1 Contexte

Dans le cadre de notre projet, nous avons été sollicité par un commercial indépendant pour développer une application web cartographique répondant à des besoins spécifiques. Ce professionnel souhaitait disposer d'un outil lui permettant de visualiser sur une carte, l'ensemble des clients dont il est responsable ainsi que les prospects potentiels. Cette demande s'inscrit dans un contexte où l'organisation et la planification des campagnes de prospection sont cruciales pour optimiser les efforts commerciaux et maximiser les opportunités de vente. Nous avons choisi ce sujet pour une raison particulière : réaliser une application opérationnelle, pouvant être directement utilisée dans un cadre professionnel. Le sujet devient donc très concret et cela nous a permis d'adopter une posture professionnalisante.

1.2 Application

L'application web cartographique se présente comme une solution adéquate pour répondre à ces besoins. Une interface cartographique permet une visualisation claire et intuitive des données géospatiales, facilitant ainsi la prise de décision et la planification des actions commerciales. Grâce à cette application, notre utilisateur peut rapidement identifier les zones géographiques à fort potentiel, constater les informations essentielles des clients et ainsi, cibler ses efforts de prospection de manière plus précise.
Pour ce projet, nous avons opté pour une page web sobre et épurée, allant directement à l'essentiel. L'objectif était de fournir une interface utilisateur simple et fonctionnelle, permettant à celui-ci de se concentrer sur ses informations sans être distrait par d'éventuels éléments superflus. En intégrant des fonctionnalités essentielles telles que le filtrage des données, l'interaction avec des popups informatifs et l'export des données filtrées, nous avons veillé à ce que l'application réponde aux nécessités de notre utilisateur, lui offrant ainsi un outil pour organiser et optimiser ses campagnes de prospection. Aussi, nous souhaiterions lui offrir la possibilité de compléter la base de données dans le futur afin de permettre une utilisation sur le long terme. Pour se faire, nous réaliserons un fichier d'instruction afin de fournir des fichiers de données avec les attributs adéquats pour le bon fonctionnement des filtres.
Tout le développement de cette application s'est effectué sur une machine virtuelle sous un système d'exploitation Linux.

2. Analyse des besoins et fonctionnalités attendues

2.1. Description des besoins utilisateurs

Cette application est principalement destinée à un commercial indépendant qui souhaite visualiser et analyser les données de ses clients ainsi que celles des prospects potentiels dans les 10 régions dont il est responsable. Il nous a demandé un outil intuitif et performant pour organiser efficacement ses campagnes de prospection et ainsi optimiser ses déplacements.

Les besoins :

- affichage interactif des données géographiques :

Les utilisateurs doivent pouvoir visualiser les clients, prospects et départements sur une carte interactive. Cela permet une compréhension rapide et visuelle de la répartition géographique et des informations des entités.

- filtrage multicritère :

Les utilisateurs doivent pouvoir filtrer les données en fonction de plusieurs critères pour affiner leur analyse. Les critères de filtrage incluent la localisation, le potentiel et le chiffre d'affaires.

2.2. Interface Homme-Machine et fonctionnalités principales

Affichage Cartographique :

- fond de carte OpenStreetMap et Esri :

L'application utilise OpenStreetMap et Esri comme fond de carte, offrant ainsi une représentation détaillée et précise des zones géographiques. OpenStreetMap a été choisi pour sa richesse en données géospatiales et sa mise à jour régulière et le fond de carte satellite Esri pour une meilleure visualisation du territoire.

- couches interactives pour les clients, prospects et départements :

Les données sont organisées en couches interactives, permettant aux utilisateurs de visualiser séparément les clients, prospects et départements. Chaque couche peut être activée ou désactivée selon les besoins de l'utilisateur. Aussi, les données ponctuelles sont représentées sous forme de marqueurs permettant le regroupement des entités proches géographiquement pour plus de lisibilité.

Filtres :

- par départements :

Les utilisateurs peuvent filtrer les données par département, ce qui permet de se concentrer sur des zones géographiques spécifiques et d'adapter les campagnes de prospection en conséquence.

- par potentialité (faible, moyen, fort, très fort) :

Les prospects peuvent être filtrés en fonction de leur potentiel commercial, permettant aux utilisateurs de prioriser leurs efforts en fonction des opportunités les plus prometteuses.

- par chiffre d’affaires (uniquement pour les clients) :

Les clients peuvent être filtrés en fonction de leur chiffre d'affaires, ce qui permet aux utilisateurs d'identifier les clients les plus rentables et d'adapter leurs stratégies commerciales en conséquence.

Interactivité :

Lorsque l'utilisateur clique sur une entité (client ou prospect) sur la carte, un popup s'affiche avec des informations détaillées, telles que le nom, l'adresse, la ville, et des commentaires.

Pour les clients, un diagramme dynamique affiche le chiffre d'affaires sur deux années, offrant ainsi une meilleure vue d'ensemble des performances commerciales.

Exportation :

- génération de fichiers .csv contenant les données filtrées :

Les utilisateurs peuvent exporter les données filtrées au format CSV. Cette fonctionnalité permet de sauvegarder les résultats des analyses pour une utilisation ultérieure ou pour les intégrer dans d'autres systèmes de gestion commerciale.

3. Architecture de l'application

3.1. Diagrammes

a. Diagramme d'architecture logique (3- tiers)

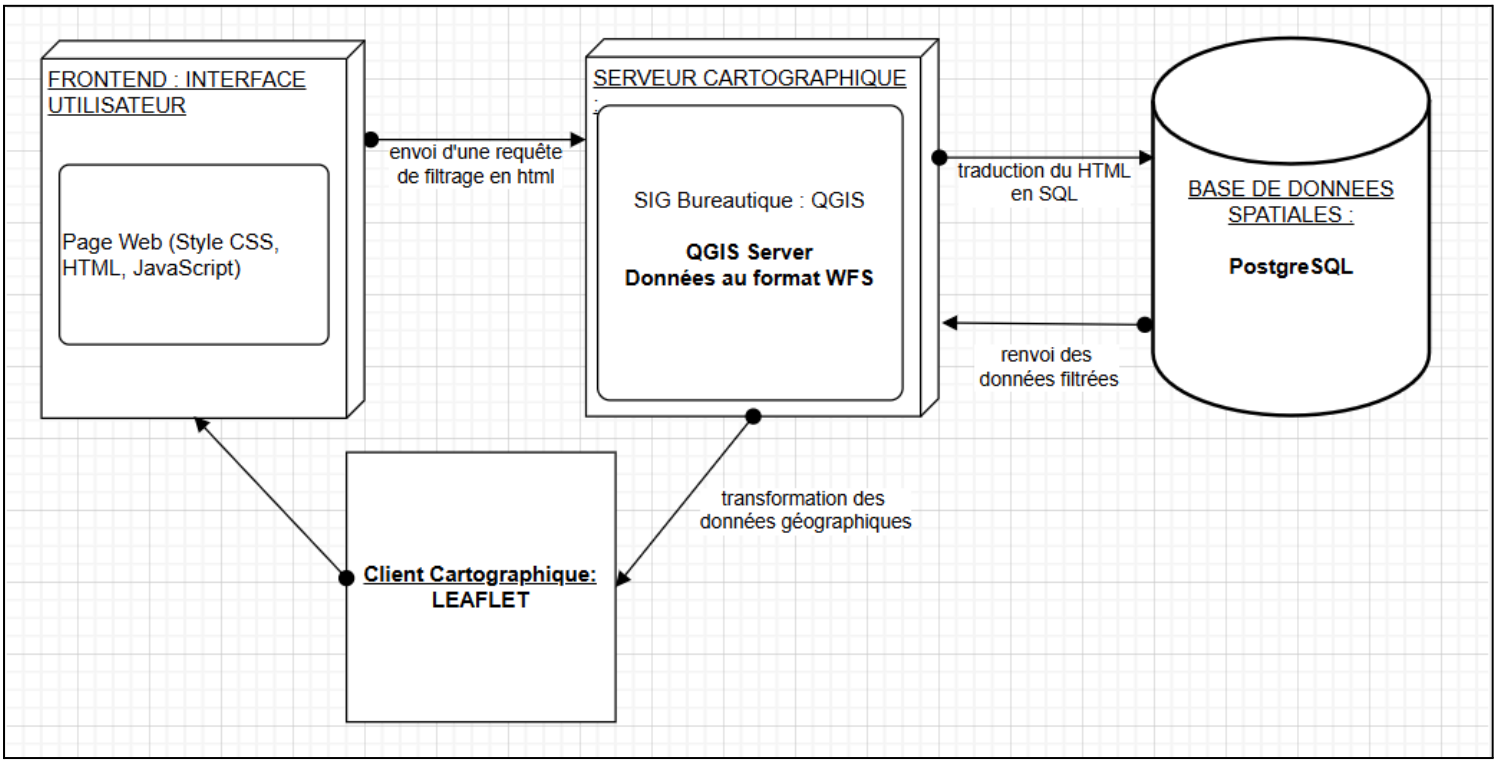

Figure 2 - Diagramme d'architecture logique de notre application en trois tiers

Notre solution cartographique repose sur une architecture trois tiers. La couche de présentation, développée en HTML, CSS et JavaScript, constitue l'interface utilisateur. Elle permet de configurer des requêtes géospatiales via un formulaire interactif et d'afficher les résultats sous forme de cartes dynamiques grâce à la bibliothèque Leaflet, qui rend les données WFS ou tuiles vectorielles utilisables en une interface interactive.

Le serveur cartographique, au cœur du traitement, utilise QGIS Server pour publier des services conformes aux *standards OGC (ensemble de normes ouvertes qui facilitent*

l'interopérabilité entre différents systèmes géospatiaux), tels que WFS et WMS. Ce serveur interprète les requêtes HTTP envoyées par le frontend, les transforme en requêtes SQL adaptées, puis effectue les traitements spatiaux nécessaires avant de transmettre les résultats au client cartographique.
Pour notre application, nous avons choisi d'utiliser le protocole WFS (Web Feature Service) pour accéder aux données géospatiales via QGIS Server. L'intérêt principal du protocole WFS est de pouvoir manipuler directement la donnée sous forme vectorielle. Ainsi, nous avons intégré les données au format geojson, nous donnant accès aux attributs de chaque table depuis l'interface utilisateur.

La base de données spatiale, basée sur PostgreSQL et son extension PostGIS, est utilisée pour stocker et manipuler des données géographiques sous forme de vecteurs comme des points, lignes ou polygones. Elle prend en charge les requêtes SQL incluant des opérations géospatiales complexes.

Grâce à cette architecture, la solution garantit une interopérabilité basée sur les standards et une robustesse adaptée aux exigences des systèmes de webmapping modernes.

b. Diagramme de flux de contrôle

Dans cette section, nous détaillons les outils et technologies sélectionnés pour la réalisation de l'application web cartographique. L'architecture de notre code JavaScript repose sur une organisation modulaire et claire, permettant une gestion fluide des données géospatiales et des interactions utilisateur. Tout commence avec la fonction **initialize()**, qui agit comme point d'entrée pour initialiser la carte interactive, charger les données GeoJSON, et configurer la logique d'exportation. Les données, qu'il s'agisse des départements ou des clients/prospects, sont récupérées via **loadGeoJSONData()**, puis filtrées dynamiquement grâce à des fonctions comme **applyClientFilter()** et **applyProspectFilter()**, conçues pour manipuler les données selon les besoins. Ces données filtrées sont ensuite rendues visuelles sur la carte via des couches GeoJSON enrichies, où les popups informatives sont créées avec **createGeoJSONLayerWithPopup()**, intégrant même des graphiques dynamiques rendus avec **renderChart()**. Enfin, la fonctionnalité d'exportation, gérée par **setupExportLogic()**, transforme les données filtrées en fichiers CSV prêts à être téléchargés grâce à **convertGeoJSONToCSV()**. L'architecture que nous avons choisie permet d'isoler chaque fonctionnalité en modules spécifiques, facilitant leur débogage, leur amélioration, et leur intégration future dans des scénarios d'usage plus complexes.

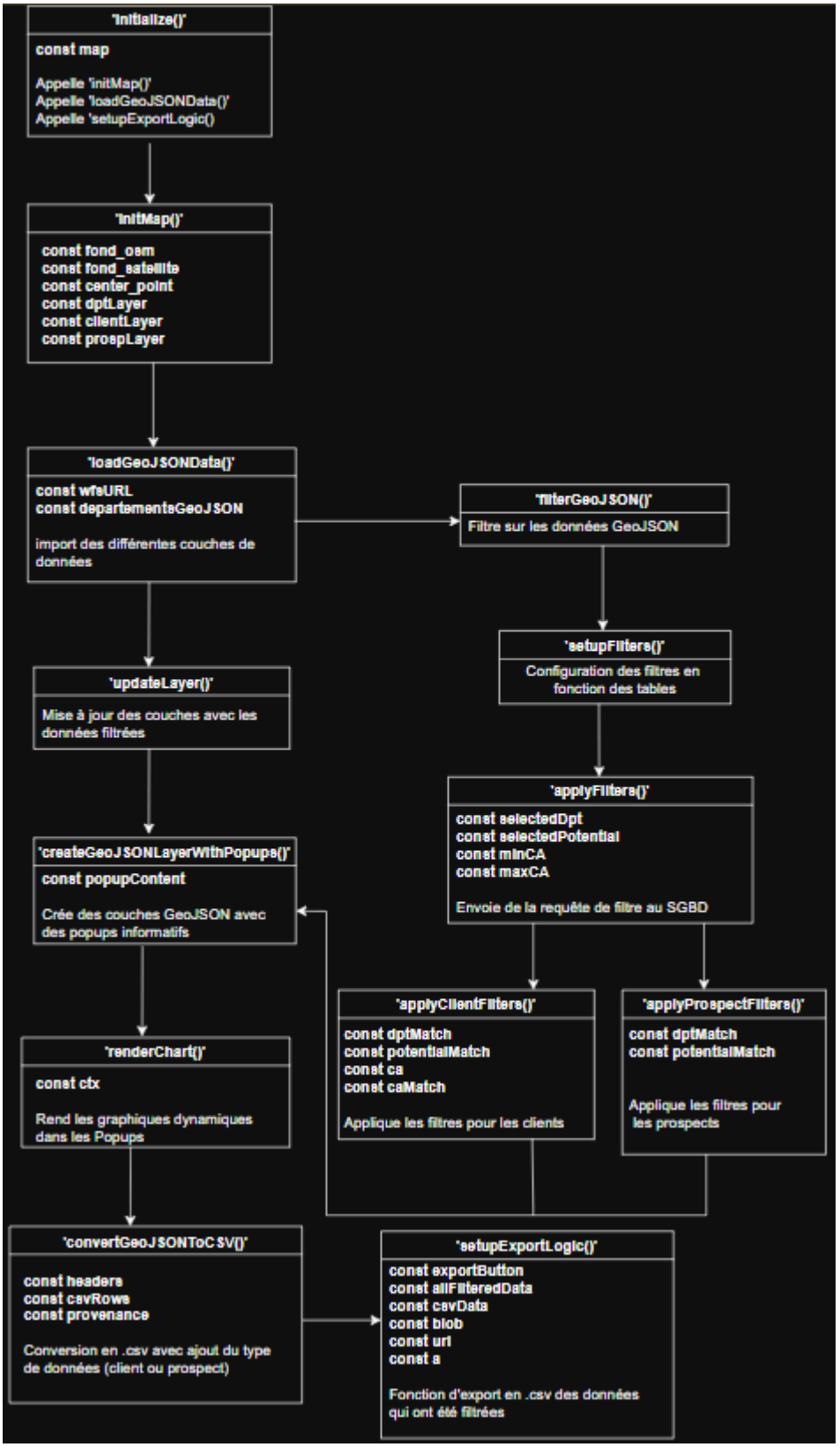

Figure 3 - Diagramme de flux de contrôle de notre application, avec les différentes fonctions expliquées

3.2. Frontend : Affichage cartographique et interactions utilisateur

Le frontend de notre solution cartographique repose sur des technologies standards liées au webmapping, conçues pour offrir une interface utilisateur intuitive et interactive, adaptée aux besoins d'un commercial indépendant.

L'interface est structurée grâce à **HTML**, qui organise les différents éléments de la page, tels que la carte interactive, les filtres, et les options d'exportation. L'esthétique et la mise en page responsive, adaptées aux différentes tailles d'écran, sont assurées par **CSS**, tandis que le script **JavaScript** prend en charge les interactions dynamiques de l'application. Ce dernier gère, par exemple, le filtrage des données en temps réel, la mise à jour des couches cartographiques, et la transmission des requêtes au serveur.

Pour la visualisation cartographique, nous avons choisi **Leaflet.js**, une bibliothèque JavaScript open source réputée pour sa légèreté et ses performances. Leaflet permet d'intégrer des fonds de carte, tels que OpenStreetMap ou Esri Satellite, de superposer des couches GeoJSON issues de notre serveur cartographique, et d'ajouter des fonctionnalités interactives, comme les marqueurs, les popups informatives, et les contrôles de navigation. Ce choix s'impose par sa simplicité d'utilisation, sa documentation complète et sa capacité d'extension, ce qui le rend parfaitement adapté à notre projet.

Enfin, pour enrichir les informations affichées dans les popups, nous utilisons **Chart.js** pour générer des graphiques dynamiques, tels que l'évolution du chiffre d'affaires d'un client. Cette bibliothèque est particulièrement efficace grâce à sa facilité d'intégration et à sa capacité à produire des visualisations claires et esthétiques, rendant les données plus accessibles et compréhensibles pour l'utilisateur.

3.3. Backend : Serveur cartographique et base de données géographique

Dans notre solution web cartographique, nous avons opté pour l'utilisation de QGIS Server en tant que serveur cartographique. QGIS Server est une extension de QGIS permettant de publier des données spatiales sous forme de services web standards, tels que WFS (Web Feature Service). Les couches vectorielles représentant les clients, les prospects, et les départements sont au format GeoJSON et intégrées directement à l'application. Ce choix nous a permis de gérer efficacement les données géographiques et de les rendre accessibles sans nécessiter le développement d'un backend complexe.

Le serveur cartographique joue un rôle central dans notre architecture trois tiers, en assurant la communication entre la base de données géographique et l'interface utilisateur. En exposant les données via des services standardisés, QGIS Server garantit une compatibilité optimale avec notre client cartographique (utilisant Leaflet) et offre une flexibilité pour les futures évolutions de la solution.
La base de données géographique constitue l'épine dorsale de notre solution, assurant la structuration et la gestion des données spatiales. Ces données répondent aux besoins

spécifiques d'un commercial indépendant, en leur permettant de visualiser, analyser, et manipuler des informations géographiques essentielles.

Les données utilisées sont stockées sous forme de couches ponctuelles et publiées via QGIS Server. Ces couches incluent des informations qui peuvent être exploitées. Les attributs varient selon les tables, ce qui permet de réaliser des requêtes croisées pertinentes. Par exemple, il est possible de croiser les données des prospects avec celles des départements pour identifier des zones géographiques à fort potentiel ou de combiner les informations sur les clients et leur chiffre d'affaire pour optimiser les actions commerciales.

Pour garantir l'actualisation des données dans la solution cartographique de manière sécurisée, les administrateurs peuvent modifier directement les tables dans la base de données via PostgreSQL/PostGIS, qui est intégrée à QGIS. Cette méthode centralisée facilite l'administration des données et permet une synchronisation immédiate avec les couches publiées sur le serveur cartographique.

Dans des projets nécessitant des traitements spatiaux complexes ou manipulant de grands volumes de données, l'utilisation de PostgreSQL/PostGIS est un atout certain. Cette solution permet de bénéficier d'une gestion avancée des géométries spatiales, d'index spatiaux pour accélérer les requêtes, et de fonctions analytiques telles que le calcul de distances, la recherche de proximité, ou les opérations topologiques.

Cette option est particulièrement justifiée pour des cas d'usage où des analyses poussées sont nécessaires. Par exemple, un commercial pourrait exploiter PostgreSQL/PostGIS pour identifier automatiquement les prospects dans un rayon précis autour des clients existants ou pour optimiser la couverture commerciale sur un territoire donné.

4. Conclusion et perspectives

4.1. Bilan du projet

Pour conclure, notre projet s'articule autour de fonctions simples à mettre en œuvre et à utiliser. L'objectif de ce projet à été tout d'abord de créer une application web cartographique correspondant à une logique d'architecture 3 tiers : interaction avec la base de données et le client cartographique via un serveur permettant aux données d'être requêtées et d'afficher le résultat voulu. Grâce à cette logique d'architecture, il est aisé de moduler selon les envies les tiers à des fins de mise à jour et de gestion.

Ensuite, il nous a paru pertinent de travailler sur un projet utile pour un professionnel indépendant afin de répondre à de réels besoins d'entreprises. Cela nous a permis de comprendre les enjeux de ce professionnel, ses attentes quant au fonctions développées, et également la volonté de ce projet de s'inscrire dans une temporalité plus longue que le cadre de nos études pour continuer à alimenter cette solution, et pouvoir l'utiliser et gérer la maintenance à posteriori de cette évaluation.

Enfin, nous souhaitions réaliser une interface simple et compréhensible par tout un chacun, surtout pour des personnes n'ayant pas de compétences spécifiques en informatique ou en

SIG et désireux de pouvoir traiter de la donnée géographique. Ce genre d'application est de plus en plus utilisée, notamment pour des applications informatives destinées au grand public (événements multi lieux, réseau de transport de communes, tourisme…).

4.2. Améliorations possibles

De nombreuses améliorations peuvent être apportées à l'application, en fonction des besoins des futurs utilisateurs. Dans le cadre des besoins actuels, nous pourrions améliorer les filtres en ajoutant un filtre par temps de trajet maximal en fonction des vitesses par tronçons, ce qui nécessiterait d'ajouter dans la base de données existantes les tronçons de la bd topo ou appeler directement une API par exemple. Aussi, dès lors qu'il y aura volonté de compléter la base de données avec de nouvelles tables, il pourrait y avoir d'autres filtres spécifiques aux attributs des tables ajoutées. Nous pourrions également créer un calcul des itinéraires optimaux permettant au professionnel d'organiser ses déplacements facilement. De plusi, nous pourrions rajouter des outils d'analyse comme des dashboards permettant aux utilisateurs de visualiser de manière simplifiée et chiffrée les données qu'ils souhaitent utiliser. Enfin, l'esthétique et la navigation pourraient être améliorées selon les goûts et envies de chacun pour une meilleure expérience.

Liens utiles :

- Documentation QGIS Serveur :

  https://docs.qgis.org/3.34/fr/docs/server_manual/index.html#index-0

- Documentation Leaflet :

  https://leafletjs.com/reference.html

Annexes :

```
HTML_web_map.html   CSS_web_map.css   JS_web_map.js
home > webmapping > Bureau > webmap > JS_web_map.js > convertGeoJSONToCSV

113
114
115  // Applique les filtres sélectionnés par l'utilisateur et met à jour les couches
116 > function applyFilters(clientLayer, prospLayer) {…
131  }
132
133
134  // Appliquer les filtres pour les clients (inclut le filtre CA)
135 > function applyClientFilters(geoJSON, selectedDpt, selectedPotential, minCA, maxCA) {…
149  }
150
151  // Appliquer les filtres pour les prospects (sans filtre CA)
152 > function applyProspectFilters(geoJSON, selectedDpt, selectedPotential) {…
164  }
165
166
167  // met à jour les couches avec les données filtrées
168 > function updateLayer(layer, geoJSON, style) {…
173  }
174
175
176  // crée des couches GeoJSON avec des popups informatifs
177 > function createGeoJSONLayerWithPopups(geoJSON, style) {…
201  }
202
203
204  // rend des graphiques dynamiques dans les popups
205 > function renderChart(canvasId, ca23, ca24) {…
227  }
228
229
230  // Conversion en .csv avec ajout du type de données (clients ou prospects) appelé dans setupExport
231 > function convertGeoJSONToCSV(features) {…
253  }
```

Figure 4 - Extrait de notre fichier JavaScript

```
HTML_web_map.html   CSS_web_map.css ●   JS_web_map.js
home > webmapping > Bureau > webmap > CSS_web_map.css > #HTML_web_map

1   /* Barre latérale */
2 > .sidebar {…
11  }
12
13  /* h3 style */
14  /* Style général pour le titre */
15 > h3 {…
21  }
22
23  /* Bouton d'application des filtres */
24 > form button {…
33  }
34
35 > form button:hover {…
38  }
39
40  /* Style du bandeau en haut */
41 > .banner {…
54  }
55
56  /* Carte */
57  #HTML_web_map {
58    position: absolute;
59    top: 80px; /* La carte commence juste après la bannière */
60    left: 250px; /* La carte commence après la barre latérale */
61    width: calc(100vw - 250px);  /* La carte occupe le reste de la largeur */
62    height: calc(100vh - 80px);  /* La carte prend le reste de la hauteur, sans chevauchement */
63  }

Restricted Mode   ⊗ 0 ⚠ 0   0
Ln 63, Col 2   Spaces: 2   UTF-8   CRLF   CSS
```

Figure 5 - Extrait du fichier de style CSS utilisé

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Carte de Prospection Commerciale</title>

    <!-- Feuilles de style Leaflet -->
    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.3.4/dist/leaflet.css" />
    <link rel="stylesheet" href="https://unpkg.com/leaflet.markercluster/dist/MarkerCluster.css" />
    <link rel="stylesheet" href="https://unpkg.com/leaflet.markercluster/dist/MarkerCluster.Default.css" />

    <!-- Feuille de style personnalisée -->
    <link rel="stylesheet" type="text/css" href="CSS_web_map.css">

    <!-- Scripts Leaflet et autres bibliothèques -->
    <script src="https://unpkg.com/leaflet@1.3.4/dist/leaflet.js"></script>
    <script src="https://unpkg.com/leaflet.markercluster/dist/leaflet.markercluster.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
    <!-- Bandeau supérieur -->
    <div class="banner">
        <h1>Carte de Prospection Commerciale</h1>
    </div>

    <!-- Barre latérale pour les filtres -->
    <div class="sidebar" id="sidebar">
        <!-- Filtrage par départements -->
        <h3>Filtrer par départements</h3>
        <form id="checkboxContainer">
            <label><input type="checkbox" value="07"> Ardèche </label><br>
            <label><input type="checkbox" value="09"> Ariège </label><br>
            <label><input type="checkbox" value="11"> Aude </label><br>
```

Figure 6 - Extrait de notre fichier HTML